

Nama	Muhammad Abyan Ridhan Siregar
NIM	1103210053
Kelas	TK-45-01

Machine Learning Task Week-14

Bidirectional RNN Model

Part 1 : Import Library

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

import re
import string
import os
```

Penjelasan

- Mengimpor berbagai pustaka yang dibutuhkan:
 - **Numpy** untuk operasi numerik dasar.
 - **Torch** (PyTorch) untuk membangun model deep learning.
 - **sklearn.model_selection** untuk membagi dataset (train/val).
 - **re, string, os** untuk keperluan *string manipulation* dan pengaturan path.

Part 2 : Data Preparation

```
def load_dataset(file_path):
    """
    Membaca file imdb_labelled.txt
    Format tiap baris: <teks>\t<label>
    """
    texts = []
    labels = []
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
```

```

# Pecah berdasarkan tab
line_split = line.strip().split('\t')
if len(line_split) == 2:
    text, label = line_split
    texts.append(text)
    labels.append(int(label))
return texts, labels

```

```

def text_cleaning(text):
    """
    Membersihkan teks sederhana: lower, hapus angka & tanda baca, dsb.
    """
    text = text.lower()
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text) # hapus tanda baca
    text = re.sub(r'\d+', '', text) # hapus angka
    text = re.sub(r'\s+', ' ', text) # hapus spasi berlebih
    text = text.strip()
    return text

```

```

def build_vocab(cleaned_texts, min_freq=1):
    """
    Membuat word2idx sederhana berdasarkan frekuensi kemunculan kata.
    min_freq = minimal frekuensi agar kata dimasukkan ke vocab
    """
    freq_dict = {}
    for text in cleaned_texts:
        for word in text.split():
            freq_dict[word] = freq_dict.get(word, 0) + 1

    # sort by frequency
    sorted_words = sorted(freq_dict.items(), key=lambda x: x[1], reverse=True)

    # Buat word2idx, sisipkan token khusus <PAD> dan <UNK>
    word2idx = {'<PAD>':0, '<UNK>':1}
    idx = 2
    for word, freq in sorted_words:
        if freq >= min_freq:
            word2idx[word] = idx

```

<pre> idx += 1 return word2idx </pre>
<pre> def encode_text(text, word2idx, max_len=50): """ Mengubah teks menjadi list of token-id (integer), dan memotong/padding sampai max_len. """ tokens = text.split() encoded = [] for token in tokens: encoded.append(word2idx.get(token, word2idx['<UNK>'])) # potong jika panjang > max_len encoded = encoded[:max_len] # padding jika panjang < max_len if len(encoded) < max_len: encoded += [word2idx['<PAD>']] * (max_len - len(encoded)) return encoded </pre>
<pre> class IMDBDataset(Dataset): def __init__(self, texts, labels, word2idx, max_len=50): self.texts = texts self.labels = labels self.word2idx = word2idx self.max_len = max_len def __len__(self): return len(self.texts) def __getitem__(self, idx): text = self.texts[idx] label = self.labels[idx] encoded_text = encode_text(text, self.word2idx, self.max_len) return torch.LongTensor(encoded_text), torch.LongTensor([label]) # (seq), (label) </pre>
Penjelasan : • load_dataset(file_path): Membaca file dataset (misalnya imdb_labelled.txt) yang berisi teks dan label, lalu menyimpannya ke dalam list texts dan labels.

- **text_cleaning(text)**: Melakukan pembersihan teks sederhana (mengubah huruf menjadi *lowercase*, menghapus tanda baca, angka, dan spasi berlebih).
- **build_vocab(cleaned_texts, min_freq=1)**: Membuat *dictionary* (*word2idx*) yang memetakan kata ke indeks. Kata yang muncul kurang dari *min_freq* dapat diabaikan.
- **encode_text(text, word2idx, max_len=50)**: Mengubah teks yang sudah dibersihkan menjadi deretan indeks (*token IDs*). Juga dilakukan pemotongan jika panjang melebihi *max_len* dan *padding* jika kurang dari *max_len*.
- **IMDBDataset(Dataset)**: Kelas khusus untuk PyTorch yang mengelola data agar siap di-*load* oleh DataLoader. Setiap item *dataset* mengembalikan sepasang (*encoded_text*, *label*) dalam bentuk *torch.Tensor*.

Part 3 :Model Definition

```
class BiRNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden_size=128, num_layers=1,
                  pooling='max', num_classes=2):
        super(BiRNNClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # RNN bidirectional
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True, # <-- Bidirectional diaktifkan
            nonlinearity='tanh',
            dropout=0.0
        )

        self.pooling = pooling # 'max' or 'avg'
        # karena bidirectional, hidden size output = hidden_size * 2
        self.fc = nn.Linear(hidden_size * 2, num_classes)

    def forward(self, x):
        # x shape: (batch_size, seq_len)
```

```

embedded = self.embedding(x) # (batch_size, seq_len, embed_dim)
rnn_out, h_n = self.rnn(embedded) # (batch_size, seq_len, hidden_size*2)

if self.pooling == 'max':
    # MaxPooling over seq_len
    pooled, _ = torch.max(rnn_out, dim=1) # (batch_size, hidden_size*2)
else:
    # AvgPooling
    pooled = torch.mean(rnn_out, dim=1) # (batch_size, hidden_size*2)

logits = self.fc(pooled) # (batch_size, num_classes)
return logits

```

1. Konstruktor (__init__)

- **vocab_size**: Jumlah total kata unik dalam kosakata, digunakan untuk menentukan ukuran matriks embedding.
- **embed_dim**: Dimensi dari vektor embedding yang merepresentasikan kata-kata.
- **hidden_size**: Jumlah unit tersembunyi dalam lapisan RNN. Karena ini adalah model bidirectional, output sebenarnya akan menjadi dua kali ukuran ini.
- **num_layers**: Jumlah lapisan RNN bertingkat yang digunakan.
- **pooling**: Metode pooling untuk mereduksi output dari seluruh urutan menjadi satu representasi tetap, baik menggunakan *max pooling* atau *average pooling*.
- **num_classes**: Jumlah kelas output untuk klasifikasi.

Komponen Utama:

1. **Embedding Layer**: Mengonversi token input (kata) menjadi representasi vektor berdimensi tetap.
2. **Bidirectional RNN**: Digunakan untuk memproses input secara maju dan mundur, sehingga menangkap konteks dari kedua arah dalam urutan input.
3. **Pooling**: Setelah output dari RNN dihasilkan, digunakan pooling (max atau average) untuk mereduksi dimensi sekuens menjadi satu vektor.
4. **Linear Layer**: Lapisan akhir untuk memetakan representasi hasil pooling ke jumlah kelas output.

2. Fungsi Forward

- **Input**: Tensor x yang merepresentasikan urutan kata-kata dalam batch (dengan ukuran $(batch_size, seq_len)$).
- **Proses**:

1. **Embedding:** Input diubah menjadi representasi embedding, menghasilkan tensor berdimensi (batch_size, seq_len, embed_dim).
 2. **RNN:** Proses embedding menggunakan RNN bidirectional, menghasilkan tensor keluaran dengan dimensi (batch_size, seq_len, hidden_size*2).
 3. **Pooling:**
 - *MaxPooling:* Memilih nilai maksimum dari setiap fitur di sepanjang dimensi urutan.
 - *AveragePooling:* Menghitung rata-rata nilai di sepanjang dimensi urutan.
 4. **Linear Layer:** Representasi hasil pooling diteruskan ke lapisan linear untuk menghasilkan logit dengan dimensi (batch_size, num_classes).
- **Output:** Logit yang menunjukkan skor untuk setiap kelas dalam proses klasifikasi.

3. Fitur Utama Model

- *Bidirectional RNN:* Memungkinkan model untuk menangkap informasi konteks secara lebih luas karena mempertimbangkan urutan maju dan mundur.
- *Pooling:* Menyediakan fleksibilitas dalam memilih bagaimana informasi urutan dirangkum.
- *Generalisasi:* Model ini cukup fleksibel untuk berbagai tugas klasifikasi teks, baik biner maupun multiklas.

Part 4 : Training & Eval Utils

```
def calculate_accuracy(y_pred, y_true):
    predicted = torch.argmax(y_pred, dim=1)
    correct = (predicted == y_true.view(-1)).sum().item()
    total = y_true.size(0)
    return correct / total

def train_one_epoch(model, dataloader, optimizer, criterion, device='cpu'):
    model.train()
    running_loss = 0.0
    running_acc = 0.0
    for x_batch, y_batch in dataloader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(x_batch)
```

```

loss = criterion(outputs, y_batch.view(-1))
loss.backward()
optimizer.step()

acc = calculate_accuracy(outputs, y_batch)
running_loss += loss.item() * x_batch.size(0)
running_acc += acc * x_batch.size(0)

epoch_loss = running_loss / len(dataloader.dataset)
epoch_acc = running_acc / len(dataloader.dataset)
return epoch_loss, epoch_acc

```

```

def validate_one_epoch(model, dataloader, criterion, device='cpu'):
    model.eval()
    running_loss = 0.0
    running_acc = 0.0
    with torch.no_grad():
        for x_batch, y_batch in dataloader:
            x_batch = x_batch.to(device)
            y_batch = y_batch.to(device)
            outputs = model(x_batch)
            loss = criterion(outputs, y_batch.view(-1))

            acc = calculate_accuracy(outputs, y_batch)
            running_loss += loss.item() * x_batch.size(0)
            running_acc += acc * x_batch.size(0)
        epoch_loss = running_loss / len(dataloader.dataset)
        epoch_acc = running_acc / len(dataloader.dataset)
    return epoch_loss, epoch_acc

```

Kode di atas berisi tiga fungsi yang digunakan dalam proses pelatihan dan validasi model neural network, khususnya untuk tugas klasifikasi.

1. calculate_accuracy

Fungsi ini menghitung akurasi prediksi model dibandingkan dengan label sebenarnya:

- **Input:**
 - y_pred: Tensor prediksi model (output logit dari lapisan terakhir model).
 - y_true: Tensor label sebenarnya.
- **Proses:**

- Prediksi kelas diambil dari nilai maksimum pada dimensi fitur (`torch.argmax`).
- Perbandingan dilakukan antara prediksi dan label sebenarnya, lalu jumlah prediksi yang benar dihitung.
- Akurasi dihitung sebagai rasio prediksi yang benar terhadap total sampel.

- **Output:** Nilai akurasi dalam bentuk float.

2. `train_one_epoch`

Fungsi ini menjalankan satu epoch pelatihan untuk model:

- **Input:**
 - `model`: Model neural network.
 - `dataloader`: Dataloader untuk memberikan batch data selama pelatihan.
 - `optimizer`: Optimizer untuk memperbarui parameter model.
 - `criterion`: Fungsi kerugian untuk menghitung loss.
 - `device`: Perangkat tempat pelatihan dilakukan (CPU atau GPU).
- **Proses:**
 - Mode pelatihan diaktifkan dengan `model.train()`.
 - Loop iterasi melalui batch data:
 - Data input dan label dipindahkan ke perangkat yang ditentukan.
 - Optimizer direset (`optimizer.zero_grad()`).
 - Forward pass dilakukan untuk mendapatkan output model.
 - Kerugian dihitung menggunakan `criterion`.
 - Backward pass dilakukan untuk menghitung gradien.
 - Optimizer memperbarui parameter model.
 - Akurasi dihitung dan metrik (loss dan akurasi) diakumulasi.
 - Setelah semua batch selesai, rata-rata loss dan akurasi dihitung berdasarkan ukuran dataset.
- **Output:** Loss dan akurasi rata-rata per epoch.

3. `validate_one_epoch`

Fungsi ini menjalankan validasi untuk model setelah pelatihan:

- **Input:** Sama seperti `train_one_epoch`.
- **Proses:**
 - Mode evaluasi diaktifkan dengan `model.eval()`.
 - Loop iterasi melalui batch data, tetapi gradien tidak dihitung (`torch.no_grad()`), untuk efisiensi dan penghematan memori.
 - Forward pass dilakukan untuk menghitung output dan loss, serta akurasi dihitung.
 - Metrik (loss dan akurasi) diakumulasi tanpa pembaruan parameter.
 - Setelah semua batch selesai, rata-rata loss dan akurasi dihitung.

- **Output:** Loss dan akurasi rata-rata per epoch.

Part 5 : Early Stopping Class

```
class EarlyStopping:
    def __init__(self, patience=5, delta=0.0):
        self.patience = patience
        self.counter = 0
        self.best_loss = None
        self.early_stop = False
        self.delta = delta

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0
```

Penjelasan

- **EarlyStopping:** Menghentikan proses training jika *loss* pada *validation set* tidak membaik setelah beberapa *epoch* tertentu (*patience*).
- **self.delta:** Toleransi perbedaan minimal agar dianggap “membaik”.
- **self.counter:** Menghitung berapa kali *validation loss* tidak mengalami penurunan.

Part 6 : Main Training Function

```
def train_and_evaluate(model,
                        train_loader,
                        val_loader,
                        optimizer,
                        scheduler,
                        criterion,
```

```

        epochs=10,
        patience=5,
        device='cpu'):

early_stopper = EarlyStopping(patience=patience)

best_val_loss = float('inf')
best_model_state = None

for epoch in range(1, epochs+1):
    # Training
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    # Validation
    val_loss, val_acc = validate_one_epoch(model, val_loader, criterion, device)

    # Scheduler step (misalnya StepLR atau lainnya)
    if scheduler is not None:
        scheduler.step(val_loss) # Jika pakai ReduceLRonPlateau, butuh param (val_loss)

    print(f"Epoch [{epoch}/{epochs}] | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

    # Cek apakah ini model terbaik
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_model_state = model.state_dict()

    # Early Stopping
    early_stopper(val_loss)
    if early_stopper.early_stop:
        print("Early stopping triggered!")
        break

# Kembalikan state terbaik
if best_model_state is not None:

```

```
model.load_state_dict(best_model_state)
return model
```

Penjelasan

- **train_and_evaluate(...):**
 1. Inisialisasi **Early Stopping**.
 2. Loop per-epoch:
 - Panggil `train_one_epoch` untuk melatih model di *train set*.
 - Panggil `validate_one_epoch` untuk mengevaluasi model di *validation set*.
 - Update *learning rate* melalui *scheduler* (misal `ReduceLROnPlateau`) berdasarkan *validation loss*.
 - Cek **Early Stopping**: jika *loss* tidak membaik setelah *patience* tertentu, hentikan training.
 3. Jika ada model dengan *val loss* terbaik, simpan *state_dict* dan kembalikan model terbaik di akhir.

Part 7 : Experiment Config

```
from google.colab import drive
import pandas as pd
from google.colab import files
drive.mount('/content/drive')

FILE_PATH = "/content/drive/MyDrive/Machine Learning/imdb_labelled.txt" # sesuaikan dengan
path dataset
BATCH_SIZE = 32
MAX_LEN = 50
EMBED_DIM = 128
DEVICE = 'cpu' # jika ingin pakai GPU: DEVICE = 'cuda' (jika tersedia)

# Hyper-parameters yang mau kita bandingkan
HIDDEN_SIZES = [64, 128]
POOLINGS = ['max', 'avg']
OPTIMIZERS = ['SGD', 'RMSProp', 'Adam']
EPOCHS_LIST = [5, 50, 100, 250, 350]

# Callback/EarlyStop param
```

```
PATIENCE = 5
```

```
results = []
```

Penjelasan

- Di sini ditentukan berbagai **parameter** untuk eksperimen, seperti:
 - **FILE_PATH**: lokasi dataset.
 - **BATCH_SIZE**: ukuran batch.
 - **MAX_LEN**: panjang sekuens maksimum setelah *padding*.
 - **HIDDEN_SIZES**: daftar opsi *hidden size* RNN yang akan dicoba.
 - **POOLINGS**: jenis pooling (max atau avg).
 - **OPTIMIZERS**: opsi optimizer (SGD, RMSProp, Adam).
 - **EPOCHS_LIST**: daftar jumlah *epoch* yang akan dicoba.
 - **PATIENCE**: kesabaran untuk *Early Stopping*.

Part 8 : Main Experiment Script

```
def main_experiment_birnn():  
    # 1. Load & preprocess  
    texts, labels = load_dataset(FILE_PATH)  
    cleaned_texts = [text_cleaning(t) for t in texts]  
    word2idx = build_vocab(cleaned_texts, min_freq=1)  
    vocab_size = len(word2idx)  
  
    # 2. Split data  
    train_texts, val_texts, train_labels, val_labels = train_test_split(  
        cleaned_texts, labels, test_size=0.2, random_state=42  
    )  
  
    # 3. Dataset & DataLoader  
    train_dataset = IMDBDataset(train_texts, train_labels, word2idx, max_len=MAX_LEN)  
    val_dataset = IMDBDataset(val_texts, val_labels, word2idx, max_len=MAX_LEN)  
  
    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)  
    val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)  
  
    # 4. Criterion
```

```

criterion = nn.CrossEntropyLoss()

# 5. Logging
results = []

# 6. Loop experiment
for hidden_size in HIDDEN_SIZES:
    for pooling in POOLINGS:
        for opt_name in OPTIMIZERS:
            for epochs in EPOCHS_LIST:
                print("=====")
                print(f'[INFO] hidden_size={hidden_size}, pooling={pooling}, "
                    f'optimizer={opt_name}, epochs={epochs}"

# 6.1 Build model
model = BiRNNClassifier(
    vocab_size=vocab_size,
    embed_dim=EMBED_DIM,
    hidden_size=hidden_size,
    num_layers=1, # jika mau lebih deep, ganti 2-3 dsb
    pooling=pooling,
    num_classes=2
).to(DEVICE)

# 6.2 Optimizer
if opt_name == 'SGD':
    optimizer = optim.SGD(model.parameters(), lr=0.01)
elif opt_name == 'RMSProp':
    optimizer = optim.RMSprop(model.parameters(), lr=0.001)
else: # Adam
    optimizer = optim.Adam(model.parameters(), lr=0.001)

# 6.3 Scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
patience=2)

```

6.4 Train & Evaluate

```
trained_model = train_and_evaluate(  
    model,  
    train_loader,  
    val_loader,  
    optimizer,  
    scheduler,  
    criterion,  
    epochs=epochs,  
    patience=PATIENCE,  
    device=DEVICE  
)
```

6.5 Cek final performance

```
final_train_loss, final_train_acc = validate_one_epoch(trained_model, train_loader,  
criterion, DEVICE)  
  
final_val_loss, final_val_acc = validate_one_epoch(trained_model,  
val_loader, criterion, DEVICE)
```

6.6 Simpan ke results

```
results.append({  
    "hidden_size": hidden_size,  
    "pooling": pooling,  
    "optimizer": opt_name,  
    "epochs": epochs,  
    "final_train_loss": final_train_loss,  
    "final_train_acc": final_train_acc,  
    "final_val_loss": final_val_loss,  
    "final_val_acc": final_val_acc  
})
```

```
print("=====\n")
```

7. Buat DataFrame & Simpan CSV

```
df_results = pd.DataFrame(results)  
df_results.to_csv('experiment_birnn_results.csv', index=False)
```

```

print("Hasil eksperimen telah disimpan dalam 'experiment_birnn_results.csv'.")

# Jika ingin mendownload otomatis (Colab):
# from google.colab import files
# files.download('experiment_birnn_results.csv')

if __name__ == "__main__":
    main_experiment_birnn()

```

1. Memuat dan Memproses Data

- **load_dataset(FILE_PATH):** Memuat dataset teks dan labelnya dari file.
- **text_cleaning:** Membersihkan teks untuk menghilangkan noise seperti tanda baca, simbol, atau kata yang tidak relevan.
- **build_vocab:** Membuat kosakata dari data teks yang telah dibersihkan, menghasilkan word2idx (pemetaan kata ke indeks).
- **vocab_size:** Jumlah total kata unik dalam kosakata.

2. Membagi Data

- Data dibagi menjadi data pelatihan (80%) dan validasi (20%) menggunakan train_test_split.

3. Dataset dan DataLoader

- **IMDBDataset:** Kelas khusus untuk memproses teks dan label ke dalam bentuk tensor yang dapat digunakan PyTorch.
- **DataLoader:** Membagi dataset menjadi batch untuk pelatihan dan validasi.

4. Kriteria Loss

- Menggunakan fungsi nn.CrossEntropyLoss untuk tugas klasifikasi biner/multi-kelas.

5. Logging

- **results:** Daftar untuk mencatat hasil eksperimen dari setiap kombinasi parameter.

6. Eksperimen Parameter

Loop iterasi dilakukan untuk menguji kombinasi parameter berikut:

- **hidden_size:** Ukuran lapisan tersembunyi dalam RNN.
- **pooling:** Metode pooling, baik max atau avg.
- **optimizer:** Optimizer untuk memperbarui parameter (SGD, RMSProp, atau Adam).
- **epochs:** Jumlah epoch pelatihan.

Langkah Utama dalam Eksperimen:

1. **Membangun Model:** Model BiRNNClassifier dibuat dengan parameter yang diatur untuk eksperimen.
2. **Mengatur Optimizer:** Memilih optimizer berdasarkan nama yang ditentukan (SGD, RMSProp, atau Adam) dan menetapkan learning rate yang sesuai.

3. **Scheduler:** Scheduler untuk menyesuaikan learning rate selama pelatihan menggunakan ReduceLROnPlateau.
4. **Pelatihan dan Evaluasi:** Fungsi `train_and_evaluate` melatih model menggunakan data pelatihan, mengevaluasi dengan data validasi, dan mendukung mekanisme early stopping berdasarkan patience.
5. **Evaluasi Akhir:** Setelah pelatihan selesai, model dievaluasi pada data pelatihan dan validasi untuk mencatat `final_train_loss`, `final_train_acc`, `final_val_loss`, dan `final_val_acc`.
6. **Menyimpan Hasil:** Hasil eksperimen dari setiap kombinasi parameter dicatat dalam daftar `results`.

7. Menyimpan dan Menyajikan Hasil

- Hasil eksperimen disimpan sebagai file CSV (`experiment_birnn_results.csv`) menggunakan `pandas.DataFrame`.

Analisis Hasil :

Dataset ini memuat hasil eksperimen terkait performa model bi-directional RNN (BiRNN) berdasarkan parameter yang bervariasi, seperti ukuran layer tersembunyi (`hidden_size`), metode pooling (`pooling`), optimizer (`optimizer`), dan jumlah epoch (`epochs`). Metrik yang dicatat mencakup kerugian dan akurasi pada data pelatihan dan validasi: `final_train_loss`, `final_train_acc`, `final_val_loss`, dan `final_val_acc`.

Analisis Data

1. Ukuran Layer Tersembunyi (`hidden_size`): Nilai ini berperan dalam menentukan kapasitas model. Perubahan ukuran layer dapat memengaruhi akurasi dan konvergensi model.
2. Metode Pooling (`pooling`): Digunakan untuk memilih informasi utama dari output BiRNN. Contohnya adalah metode max.
3. Optimizer: Metode optimasi seperti SGD digunakan untuk memperbarui bobot jaringan selama pelatihan.
4. Epochs: Jumlah iterasi pelatihan. Bertambahnya jumlah epoch tampaknya menurunkan `final_train_loss` namun tidak selalu meningkatkan `final_val_acc`, menunjukkan potensi overfitting.
5. Hasil Akhir:
 - Akurasi pelatihan (`final_train_acc`) meningkat dengan lebih banyak epoch, tetapi peningkatan pada akurasi validasi (`final_val_acc`) terbatas.

- Kerugian validasi (final_val_loss) bervariasi, yang menunjukkan bahwa performa validasi tidak selalu sejajar dengan pelatihan.

Kesimpulan :

Eksperimen ini menunjukkan bahwa pemilihan parameter seperti hidden_size, metode pooling, dan jumlah epoch memiliki dampak signifikan terhadap performa model. Namun, peningkatan jumlah epoch melampaui titik tertentu berpotensi menyebabkan overfitting, di mana akurasi pelatihan jauh lebih tinggi dibandingkan akurasi validasi. Optimizer seperti SGD cenderung lambat dalam mencapai konvergensi, terutama untuk epoch lebih sedikit. Untuk performa yang optimal, kombinasi parameter harus dievaluasi lebih lanjut menggunakan validasi silang atau eksperimen tambahan.